

EXPERIMENT - 6

AIM

To create a stored procedure `transfer_employee(emp_id, new_dept_id)` with validation and error handling, implement triggers for salary validation and audit logging, and test important edge cases.

OBJECTIVES

- To create a stored procedure for transferring an employee between departments.
- To validate employee and department identifiers before performing a transfer.
- To use transaction control and error handling inside a stored procedure.
- To validate salary values using a trigger before INSERT or UPDATE.
- To maintain an audit log whenever an employee salary is changed.
- To test successful and invalid transfer and salary-update cases.

DATABASE USED

The queries use the CompanyDB database and the Employee–Department–Project schema. Department names and employee names have been changed from the reference data; IDs remain unchanged so the existing relationships continue to work.

USE CompanyDB;

SOURCE CODE / SQL QUERIES

1. SALARY AUDIT TABLE

Create an audit table to store old and new salary values whenever an employee salary changes.

```
CREATE TABLE IF NOT EXISTS Employee_Salary_Audit (      audit_id INT
AUTO_INCREMENT PRIMARY KEY,
      emp_id INT NOT NULL,      old_salary DECIMAL(10,2),      new_salary
DECIMAL(10,2),
      changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
      action VARCHAR(30)
);
```

2. SALARY VALIDATION TRIGGER

Reject salary values outside the allowed range of 30,000 to 1,50,000.

```
DELIMITER //
CREATE TRIGGER validate_employee_salary
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
```

```

        IF NEW.salary < 30000 OR NEW.salary > 150000 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Salary must be between 30000 and 150000';
        END IF;
    END//
DELIMITER ;

```

3. SALARY UPDATE VALIDATION TRIGGER

Validate salary changes before an existing Employee row is updated.

```

UPDATE Employee
SET salary = 20000
WHERE emp_id = 6;

```

Valid Update:

```

UPDATE Employee
SET salary = 85000
WHERE emp_id = 6;

```

I/O
AI Agent

Error

STDIN

Input for the program (Optional)

Output:

ERROR 1644 (45000) at line 118: Salary must be between 30000 and 150000

🐞 [Debug this error](#)

SALARY AUDIT TRIGGER

Store the old and new salary values after a successful salary update.

Trigger code:

```

DELIMITER //
CREATE TRIGGER audit_salary_update
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.salary <> NEW.salary THEN
        INSERT INTO Employee_Salary_Audit
        (emp_id, old_salary, new_salary, action)
        VALUES
        (NEW.emp_id, OLD.salary, NEW.salary, 'SALARY UPDATE');
    END IF;
END//
DELIMITER ;

```

CODE:

```
DELIMITER //
-- Test salary update UPDATE Employee
SET salary = 51000
WHERE emp_id = 31;
-- Display salary audit record
SELECT audit_id, emp_id, old_salary, new_salary,
       changed_at, action FROM Employee_Salary_Audit
ORDER BY audit_id;
DELIMITER ;
```

Output

I/O AI Agent 121 ms

STDIN

Input for the program (Optional)

Output:

audit_id	emp_id	old_salary	new_salary	changed_at	action
1	31	50000.00	51000.00	2026-09-22 11:32:30	SALARY UPDATE

4. STORED PROCEDURE – TRANSFER EMPLOYEE

Transfer an employee to a new department after validating both IDs. The transaction is rolled back if an SQL exception occurs.

```
DELIMITER //
CREATE PROCEDURE transfer_employee(
    IN p_emp_id INT,
    IN p_new_dept_id INT
)
BEGIN
    DECLARE v_emp_count INT DEFAULT 0;
    DECLARE v_dept_count INT DEFAULT 0;
    DECLARE v_old_dept_id INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;
    START TRANSACTION;
    SELECT COUNT(*) INTO v_emp_count
    FROM Employee
    WHERE emp_id = p_emp_id;
    IF v_emp_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee does not exist';
    END IF;
    SELECT COUNT(*) INTO v_dept_count
    FROM Department
    WHERE dept_id = p_new_dept_id;
    IF v_dept_count = 0 THEN
        SIGNAL SQLSTATE '45000'
```

```

        SET MESSAGE_TEXT = 'Department does not exist';
    END IF;
    SELECT dept_id INTO v_old_dept_id
    FROM Employee
    WHERE emp_id = p_emp_id;
    IF v_old_dept_id = p_new_dept_id THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee is already in this department';
    END IF;
    UPDATE Employee
    SET dept_id = p_new_dept_id
    WHERE emp_id = p_emp_id;
    COMMIT;
END//
DELIMITER ;

```

5. TEST SUCCESSFUL TRANSFER

Transfer employee 6 from Development to Sales.

```

CALL transfer_employee(6, 4);
SELECT e.emp_id, e.emp_name,
       d.dept_name
FROM Employee e
JOIN Department d ON e.dept_id = d.dept_id
WHERE e.emp_id = 6;

```

Output

I/O	AI Agent	95 ms ✓
STDIN		
Input for the program (Optional)		
Output:		
<pre> +-----+-----+-----+ emp_id emp_name dept_name +-----+-----+-----+ 6 Vihaan Sales +-----+-----+-----+ </pre>		

The procedure validates the employee and department, updates the department and commits the transaction.

6. TEST EDGE CASES

```

-- Invalid employee
CALL transfer_employee(99, 4);

-- Invalid department
CALL transfer_employee(6, 99);

-- Same department
CALL transfer_employee(6, 4);

```

Output

queries.sql

MySQL

Run

AI Agent

Error

```

154 BEGIN
175
176 SELECT COUNT(*) INTO v_dept_count
177 FROM Department
178 WHERE dept_id = p_new_dept_id;
179
180 IF v_dept_count = 0 THEN
181     SIGNAL SQLSTATE '45000'
182     SET MESSAGE_TEXT = 'Department does not exist';
183 END IF;
184
185 SELECT dept_id INTO v_old_dept_id
186 FROM Employee
187 WHERE emp_id = p_emp_id;
188
189 IF v_old_dept_id = p_new_dept_id THEN
190     SIGNAL SQLSTATE '45000'
191     SET MESSAGE_TEXT = 'Employee is already in this department';
192 END IF;
193
194 UPDATE Employee
195 SET dept_id = p_new_dept_id
196 WHERE emp_id = p_emp_id;
197
198 COMMIT;
199 END//
200 DELIMITER ;
201
202 CALL transfer_employee(6, 4);
203
204 -- Same department
205 CALL transfer_employee(6, 4);
206
207
208 SELECT e.emp_id, e.emp_name,
209        d.dept_name
210 FROM Employee e
211 JOIN Department d ON e.dept_id = d.dept_id
212 WHERE e.emp_id = 6;
213
214
215
216

```

SQL Agent

Input for the program (Optional)

Output:

ERROR 1644 (45000) at line 207: Employee is already in this department

Debug this error

queries.sql

MySQL

Run

AI Agent

Error

```

154 BEGIN
175
176 SELECT COUNT(*) INTO v_dept_count
177 FROM Department
178 WHERE dept_id = p_new_dept_id;
179
180 IF v_dept_count = 0 THEN
181     SIGNAL SQLSTATE '45000'
182     SET MESSAGE_TEXT = 'Department does not exist';
183 END IF;
184
185 SELECT dept_id INTO v_old_dept_id
186 FROM Employee
187 WHERE emp_id = p_emp_id;
188
189 IF v_old_dept_id = p_new_dept_id THEN
190     SIGNAL SQLSTATE '45000'
191     SET MESSAGE_TEXT = 'Employee is already in this department';
192 END IF;
193
194 UPDATE Employee
195 SET dept_id = p_new_dept_id
196 WHERE emp_id = p_emp_id;
197
198 COMMIT;
199 END//
200 DELIMITER ;
201
202
203
204 -- Invalid employee
205 CALL transfer_employee(99, 4);
206
207
208
209
210 SELECT e.emp_id, e.emp_name,
211        d.dept_name
212 FROM Employee e
213 JOIN Department d ON e.dept_id = d.dept_id
214 WHERE e.emp_id = 6;
215
216

```

SQL Agent

Input for the program (Optional)

Output:

ERROR 1644 (45000) at line 205: Employee does not exist

Debug this error

```

154 BEGIN
155
156 SELECT COUNT(*) INTO v_dept_count
157 FROM Department
158 WHERE dept_id = p_new_dept_id;
159
160 IF v_dept_count = 0 THEN
161     SIGNAL SQLSTATE '45000'
162     SET MESSAGE_TEXT = 'Department does not exist';
163 END IF;
164
165 SELECT dept_id INTO v_old_dept_id
166 FROM Employee
167 WHERE emp_id = p_emp_id;
168
169 IF v_old_dept_id = p_new_dept_id THEN
170     SIGNAL SQLSTATE '45000'
171     SET MESSAGE_TEXT = 'Employee is already in this department';
172 END IF;
173
174 UPDATE Employee
175 SET dept_id = p_new_dept_id
176 WHERE emp_id = p_emp_id;
177
178 COMMIT;
179
180 END//
181 DELIMITER ;
182
183 -- Invalid department
184 CALL transfer_employee(6, 99);
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210 SELECT e.emp_id, e.emp_name,
211        d.dept_name
212 FROM Employee e
213 JOIN Department d ON e.dept_id = d.dept_id
214 WHERE e.emp_id = 6;
215
216

```

STDIN
Input for the program (Optional)

Output:
ERROR 1644 (45000) at line 204: Department does not exist
Debug this error

7. TEST SALARY VALIDATION

Attempt to insert or update a salary outside the allowed range.

```

UPDATE Employee
SET salary = 20000
WHERE emp_id = 6;
UPDATE Employee
SET salary = 85000
WHERE emp_id = 6;

```

Output

```

154 BEGIN
155
156 SELECT dept_id INTO v_old_dept_id
157 FROM Employee
158 WHERE emp_id = p_emp_id;
159
160 IF v_old_dept_id = p_new_dept_id THEN
161     SIGNAL SQLSTATE '45000'
162     SET MESSAGE_TEXT = 'Employee is already in this department';
163 END IF;
164
165 UPDATE Employee
166 SET dept_id = p_new_dept_id
167 WHERE emp_id = p_emp_id;
168
169 COMMIT;
170
171 END//
172 DELIMITER ;
173
174 DROP TRIGGER IF EXISTS validate_employee_salary_update;
175
176 DELIMITER //
177
178 CREATE TRIGGER validate_employee_salary_update
179 BEFORE UPDATE ON Employee
180 FOR EACH ROW
181 BEGIN
182     IF NEW.salary < 30000 OR NEW.salary > 150000 THEN
183         SIGNAL SQLSTATE '45000'
184         SET MESSAGE_TEXT = 'Salary must be between 30000 and 150000';
185     END IF;
186 END//
187
188 DELIMITER ;
189
190 UPDATE Employee
191 SET salary = 20000
192 WHERE emp_id = 6;
193

```

STDIN
Input for the program (Optional)

Output:
ERROR 1644 (45000) at line 218: Salary must be between 30000 and 150000
Debug this error

8. TEST SALARY AUDIT LOG

Display the audit entries created by successful salary changes.

```
SELECT audit_id, emp_id, old_salary, new_salary,  
       changed_at, action FROM Employee_Salary_Audit  
ORDER BY audit_id;
```

Output

I/O

AI Agent

98 ms

STDIN

Input for the program (Optional)

Output:

audit_id	emp_id	old_salary	new_salary	changed_at	action
1	31	50000.00	51000.00	2026-09-22 12:38:17	SALARY UPDATE

RESULT

The transfer_employee procedure was created with validation, transaction control and error handling. Salary validation and audit triggers were implemented and edge cases were tested.

CONCLUSION

Stored procedures centralize controlled database operations, while triggers automatically enforce validation rules and maintain an audit trail for important changes.